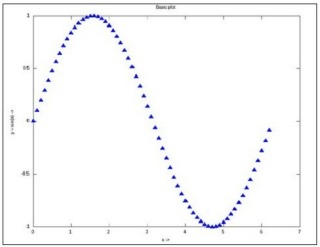

Figure 1: Basic plotting in Octave

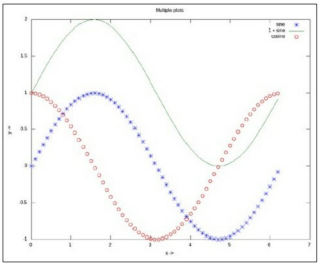

Figure 2: Multiple plots on the same axis

Advanced 2D figures

Now, if we want to have multiple graphs on the same sheet but with different axes, here is how to do it:

```
octave:1> t = 0:0.1:6*pi;
octave:2> subplot(2, 1, 1);
octave:3> plot(t, 325 * sin(t));
octave:4> xlabel("t (sec)");
octave:5> ylabel("V_{ac} (V)");
octave:6> title("AC voltage curve");
octave:7> grid("on");
octave:8> subplot(2, 1, 2);
octave:9> plot(t, 5 * cos(t));
octave:10> xlabel("t (sec)");
octave:11> ylabel("I_{ac} (A)");
octave:12> title("AC current curve");
octave:13> grid("on");
octave:14> print("-dpng", "multiple_plots_on_a_sheet.png");
octave:15>
```

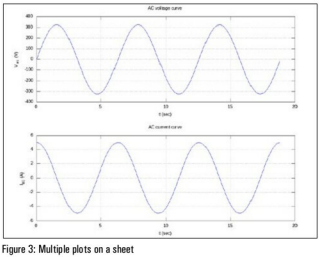

Figure 3: Multiple plots on a sheet

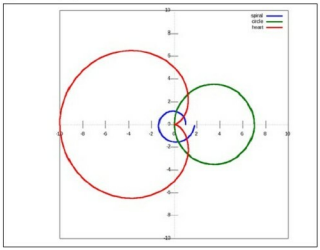

Figure 4: Polar plots

Note the usage of *subplot()*, taking the matrix dimensions (row and column) and the plot number to create the matrix of plots. In the example above, it created a 2x1 matrix of plots. As add-ons, we have used the *grid("on")* to show up the dotted grid lines, and *print()* to save the generated figure as a *.png* file.

It is not always easy to plot everything in Cartesian co-ordinates, or rather, many things are easier to plot in polar co-ordinates, e.g., a spiral, circle, heart, etc. The following code shows a few such examples. Shown alongwith is a technique of modifying the figure properties, after drawing the figure using the *set()* function. Here, it modifies the line's thickness.

```
octave:1> th = 0:0.1:2*pi;
octave:2> r1 = 1.1 .^ th;
octave:3> r2 = 7 * cos(th);
octave:4> r3 = 5 * (1 - cos(th));
octave:5> r = [r1; r2; r3];
octave:6> ph = polar(th, r, "-");
octave:7> set(ph, "LineWidth", 4);
octave:8> legend("spiral", "circle", "heart");
octave:9>
```